

Casos de Teste — BetManager

Estrutura dos Testes

```
tests/
├── unit/                                # Testes unitários (sem banco, sem IA)
│   ├── calculations.test.ts
│   ├── validators.test.ts
│   └── aiParser.test.ts
├── integration/                        # Testes de integração (banco real, IA mockada)
│   ├── bets.test.ts
│   ├── dashboard.test.ts
│   ├── stats.test.ts
│   └── goals.test.ts
└── e2e/                                # Testes end-to-end (Playwright)
    ├── betRegistration.spec.ts
    ├── aiUpload.spec.ts
    └── dashboard.spec.ts
```

Ferramentas:

- Backend: `Jest` + `Supertest`
- Banco de testes: PostgreSQL separado (`betmanager_test`)
- Mock da IA: `jest.mock` para `@anthropic-ai/sdk`
- Frontend E2E: `Playwright`

Testes Unitários

TU-01 — Cálculo de Lucro

Módulo: `src/services/calculations.ts`

ID	Descrição	Entrada	Esperado
TU-01-01	Lucro positivo (ganhou)	payout=35.00, wagered=20.00	profit=15.00
TU-01-02	Lucro negativo (perdeu)	payout=0.00, wagered=20.00	profit=-20.00
TU-01-03	Aposta nula (void)	payout=20.00, wagered=20.00	profit=0.00
TU-01-04	Cálculo payout via odd	wagered=20.00, odds=1.75	payout=35.00
TU-01-05	Odd com muitas casas decimais	wagered=28.40, odds=1.53	payout=43.45
TU-01-06	Valor alto sem erro de arredondamento	wagered=500.00, odds=2.33	payout=1165.00

```
// Exemplo de implementação do teste
describe('calculateProfit', () => {
  it('retorna lucro positivo quando ganhou', () => {
    expect(calculateProfit(35.00, 20.00)).toBe(15.00)
  })

  it('retorna lucro negativo quando perdeu', () => {
    expect(calculateProfit(0.00, 20.00)).toBe(-20.00)
  })

  it('calcula payout a partir da odd', () => {
    expect(calculatePayout(20.00, 1.75)).toBe(35.00)
  })
})
```

TU-02 — Validação de Campos

Módulo: `src/validators/betSchema.ts` (Zod)

ID	Descrição	Entrada	Esperado
TU-02-01	Data válida	"2024-09-06"	Aceita
TU-02-02	Data futura rejeitada	amanhã	ValidationError
TU-02-03	Valor apostado zero rejeitado	amount_wagered=0	ValidationError
TU-02-04	Valor apostado negativo rejeitado	amount_wagered=-10	ValidationError
TU-02-05	Odd abaixo de 1.0 rejeitada	odds=0.5	ValidationError
TU-02-06	Odd nula aceita (campo opcional)	odds=null	Aceita
TU-02-07	Resultado inválido rejeitado	result="maybe"	ValidationError
TU-02-08	Resultado válido aceito	result="won"	Aceita
TU-02-09	Descrição vazia rejeitada	description=""	ValidationError
TU-02-10	Tipo de aposta inválido rejeitado	bet_type="triple"	ValidationError

TU-03 — Parser de Resposta da IA

Módulo: `src/services/aiService.ts`

ID	Descrição	Esperado
TU-03-01	JSON válido com todas as apostas	Array com N apostas parseadas
TU-03-02	JSON com campo null (campo não detectado)	Campo como null, sem erro
TU-03-03	JSON malformado (IA retornou texto fora do JSON)	Error lançado com log
TU-03-04	Array vazio retornado	Array vazio sem exceção
TU-03-05	Casa não reconhecida retornada pela IA	bookmakerId=null, flag de aviso
TU-03-06	Data no formato DD/MM convertida corretamente	Date com ano atual inferido
TU-03-07	Valores monetários com vírgula (R\$ 20,00)	number 20.00

TU-04 — Cálculo de Estatísticas

ID	Descrição	Entrada	Esperado
TU-04-01	Hit rate com ganhos e perdas	won=6, lost=4	60.0%
TU-04-02	Hit rate sem perdas (100%)	won=5, lost=0	100.0%
TU-04-03	Hit rate sem ganhos (0%)	won=0, lost=5	0.0%
TU-04-04	Hit rate sem apostas (divisão por zero)	won=0, lost=0	None ou 0.0
TU-04-05	ROI calculado corretamente	profit=150, wagered=600	25.0%
TU-04-06	ROI negativo	profit=-100, wagered=500	-20.0%
TU-04-07	Lucro acumulado diário correto	[+15, -20, +35]	[15, -5, 30]

Testes de Integração (API)

TI-01 — CRUD de Apostas

ID	Endpoint	Cenário	Status	Response
TI-01-01	POST /bets	Aposta válida	201	Objeto da aposta criada
TI-01-02	POST /bets	Campo obrigatório ausente	422	Detalhe do campo faltando
TI-01-03	POST /bets	Data futura	422	Mensagem de validação
TI-01-04	POST /bets	bookmaker_id inexistente	404	BOOKMAKER_NOT_FOUND
TI-01-05	POST /bets	Valor apostado = 0	422	Mensagem de validação
TI-01-06	GET /bets	Sem filtros	200	Lista paginada
TI-01-07	GET /bets	Filtro por resultado "won"	200	Apenas apostas ganhas
TI-01-08	GET /bets	Filtro por período	200	Apostas no intervalo
TI-01-09	GET /bets	Busca por texto na descrição	200	Apostas com o texto
TI-01-10	GET /bets	Página 2 com per_page=10	200	Items 11-20
TI-01-11	GET /bets/{id}	ID existente	200	Objeto da aposta
TI-01-12	GET /bets/{id}	ID inexistente	404	BET_NOT_FOUND
TI-01-13	PUT /bets/{id}	Atualizar resultado	200	Aposta atualizada
TI-01-14	PATCH /bets/{id}/result	Resultado rápido	200	Apenas result e profit
TI-01-15	DELETE /bets/{id}	ID existente	200	Mensagem de confirmação
TI-01-16	DELETE /bets/{id}	ID inexistente	404	BET_NOT_FOUND

TI-02 — Batch (múltiplas apostas)

ID	Endpoint	Cenário	Status	Response
TI-02-01	POST /bets/batch	3 apostas válidas	201	<code>created: 3</code>
TI-02-02	POST /bets/batch	Array vazio	400	Erro de validação
TI-02-03	POST /bets/batch	Uma aposta inválida no lote	422	Erro com índice da aposta
TI-02-04	POST /bets/batch	50 apostas (lote grande)	201	<code>created: 50</code>

TI-03 — Extração por IA (com mock)

A Claude API é mockada nos testes de integração para evitar custos e dependência de rede.

ID	Cenário	Mock retorna	Status	Esperado
TI-03-01	Upload de PNG válido	JSON com 3 apostas	200	3 apostas no response
TI-03-02	Upload de JPG válido	JSON com 1 aposta	200	1 aposta no response
TI-03-03	Arquivo acima de 10MB	(não chama IA)	422	<code>FILE_TOO_LARGE</code>
TI-03-04	Arquivo PDF (formato inválido)	(não chama IA)	422	<code>UNSUPPORTED_FILE_TYPE</code>
TI-03-05	IA retorna JSON vazio	<code>{ "apostas": [] }</code>	200	<code>bets_detected: 0</code>
TI-03-06	IA retorna erro (timeout simulado)	Exception	500	<code>AI_API_ERROR</code>
TI-03-07	IA retorna JSON malformatado	texto sem JSON	500	<code>AI_API_ERROR</code>
TI-03-08	Campo <code>model=sonnet</code> especificado	JSON com 2 apostas	200	<code>model_used: claude-sonnet-4</code>

TI-04 — Dashboard

ID	Endpoint	Cenário	Status	Verificação
TI-04-01	GET / dashboard	period=month com dados	200	summary.total_bets correto
TI-04-02	GET / dashboard	period=month sem apostas	200	summary zerado, pending vazio
TI-04-03	GET / dashboard	Lucro acumulado calculado certo	200	profit_chart correto
TI-04-04	GET / dashboard	Apostas pendentes listadas	200	pending_bets não vazio
TI-04-05	GET / dashboard	Meta do mês exibida	200	goal.progress_pct correto
TI-04-06	GET / dashboard	period=day	200	Apenas apostas de hoje
TI-04-07	GET / dashboard	period=year	200	Apostas do ano atual

TI-05 — Estatísticas

ID	Endpoint	Verificação
TI-05-01	GET /stats/sports	Hit rate por esporte calculado corretamente
TI-05-02	GET /stats/sports	Apenas apostas com resultado (não pendentes)
TI-05-03	GET /stats/bookmakers	Lucro total por casa correto
TI-05-04	GET /stats/bet-types	ROI de simples e combinadas calculados
TI-05-05	GET /stats/bet-types	Recommendation gerada corretamente
TI-05-06	GET /stats/monthly	12 meses retornados (ou menos se sem dados)

TI-06 — Metas

ID	Endpoint	Cenário	Status	Verificação
TI-06-01	POST /goals	Meta válida	201	Meta criada
TI-06-02	POST /goals	Duplicata (mesmo mês/ano)	400	GOAL_DUPLICATE
TI-06-03	POST /goals	target_profit <= 0	422	Erro de validação
TI-06-04	GET /goals	Listar metas com progresso	200	progress_pct correto
TI-06-05	PUT /goals/{id}	Atualizar valor da meta	200	Meta atualizada
TI-06-06	DELETE /goals/{id}	Excluir meta	200	Confirmação

Testes E2E (Playwright)

TE-01 — Registro de Aposta Manual

Cenário: Usuário registra aposta manualmente com sucesso

Dado que o sistema está rodando

Quando o usuário acessa /apostas/nova

E clica em "Registro Manual"

E preenche: data=hoje, descrição="Brasil 02.5", casa="Bet365", valor=20, odd=1.75, resultado=ganhou

E clica em "Salvar Aposta"

Então vê mensagem de sucesso

E a aposta aparece no histórico

E o dashboard mostra o lucro atualizado

TE-02 — Upload de Print via IA

Cenário: Usuário faz upload de print e confirma apostas

Dado que o sistema está rodando

Quando o usuário acessa /apostas/nova

E clica em "Enviar Print"

E faz upload de uma imagem válida

Então vê o spinner "Analisando imagem..."

E vê o formulário pré-preenchido com as apostas detectadas

Quando confirma as apostas

Então vê mensagem de sucesso

E as apostas aparecem no histórico

TE-03 — Atualização Rápida de Resultado Pendente

Cenário: Usuário atualiza resultado de aposta pendente no dashboard

Dado que existe uma aposta com resultado "pendente"

Quando o usuário acessa o dashboard

E vê a aposta na seção "Apostas Pendentes"

E clica no ícone ✓ (ganhou)

Então o resultado é atualizado para "won"

E o lucro no dashboard é recalculado instantaneamente

TE-04 — Filtro de Histórico

Cenário: Usuário filtra histórico por resultado

Dado que existem apostas de diferentes resultados

Quando o usuário acessa /apostas

E seleciona o filtro resultado="won"

Então apenas apostas ganhas são exibidas

E o totalizador reflete apenas essas apostas

TE-05 — Dashboard com Período

Cenário: Usuário alterna período do dashboard

Dado que existem apostas em diferentes datas

Quando o usuário seleciona "Semana" no dashboard

Então os cards de lucro refletem apenas a semana atual

E o gráfico mostra apenas os últimos 7 dias

Testes de Regressão

Executar após cada deploy / merge:

ID	Escopo	Automático
TR-01	Todos os testes unitários	Sim (CI)
TR-02	Todos os testes de integração	Sim (CI)
TR-03	TE-01 (registro manual)	Sim (CI)
TR-04	TE-03 (atualização de pendente)	Sim (CI)
TR-05	TE-02 (upload IA)	Manual

Configuração do Ambiente de Testes

```

# conftest.py
import pytest
from httpx import AsyncClient
from app.main import app
from app.database import get_db, engine
from app.models import Base

@pytest.fixture(scope="session", autouse=True)
async def setup_test_db():
    """Cria tabelas no banco de testes antes dos testes."""
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    yield
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)

@pytest.fixture
async def client():
    async with AsyncClient(app=app, base_url="http://test") as ac:
        yield ac

@pytest.fixture
def mock_ai_response():
    """Mock padrão para a Claude API."""
    return {
        "apostas": [
            {
                "data": "06/09",
                "descricao": "Brasil 02.5 HT",
                "valor_apostado": 20.00,
                "casa": "Superbet",
                "odd": 1.75,
                "retorno_total": 35.00,
                "resultado": "won"
            }
        ]
    }

```

Critérios de Aceite (Definition of Done)

Uma funcionalidade está pronta quando:

- ☐ Todos os casos de teste unitários passam
- ☐ Todos os casos de integração da feature passam
- ☐ Teste E2E do fluxo principal passa
- ☐ Nenhuma regressão nos testes existentes
- ☐ Coverage mínimo de 80% no módulo alterado
- ☐ Tratamento de erro implementado e testado